

JALabs_SymSearch — Technical Documentation

Developer-facing internals: architecture, how the module hooks into the Magento + Mirasvit + OpenSearch search stack, the data model, every extension point, the request/index data flows, the failure model, and the environment-specific issues encountered. For configuration and operations see [USER-GUIDE.md](#).

- **Module:** `JALabs_SymSearch` (`app/code/JALabs/SymSearch`)
- **Platform:** Magento Commerce 2.4.8-p2, PHP 8.4
- **Search stack:** Mirasvit Search Ultimate 2.6.15 over OpenSearch 3.3.2 (`opensearch-knn` , `opensearch-neural-search`)
- **Integration rule:** plugins / DI only. No core or Mirasvit files are modified, so both stay upgrade-safe.

1. The problem and the chosen approach

Keyword search (BM25) can't match a query to a product unless they share tokens. That fails natural-language queries, paraphrases, and cross-language queries. The fix is **vector embeddings**: map every product and every query into the same high-dimensional space where semantic similarity $\approx$ geometric proximity, then retrieve nearest neighbours.

Three ways to graft that onto Magento were considered:

Approach	Verdict
A. Native OpenSearch hybrid — store a vector field in the <i>existing</i> product index, issue a <code>hybrid</code> (BM25 + k-NN) query	Chosen. No new infra; facets / layered nav / stock filters / Mirasvit boosts keep working because the keyword sub-query is untouched.
B. Re-rank layer — keep keyword retrieval, reorder the top N by vector similarity	Rejected: can't rescue zero-result or cross-language queries — it only reorders what keyword already found.
C. External vector DB / SaaS	Rejected: new infrastructure, a second source of truth, and facet counts / layered nav would have to be rebuilt.

The whole design follows from approach A: the vector lives **inside the Mirasvit-managed product index**, and the query becomes a **hybrid** query so everything downstream of search (aggregations, layered navigation, pagination, result rendering) is unchanged.

2. Where it plugs into the search stack

Magento's storefront quick search flows through these layers; SymSearch attaches at four of them (→). Mirasvit also plugs in here — order matters.

```

Storefront search request (quick_search_container)
|
▼
Magento\Search\Model\Search → request built from search_request.xml
|
▼
Magento\Framework\Search\SearchEngine
|
▼
Magento\OpenSearch\SearchAdapter\Adapter::query()
|   builds the query body via the Mapper, sends to OpenSearch,
|   and *swallows any error → empty result* (see §9)
|
▼
Magento\OpenSearch\SearchAdapter\Mapper::buildQuery()
|   └─ delegates to Magento\Elasticsearch\ElasticAdapter\...\Mapper
|       └─ Mirasvit ElasticsearchAddScriptToSearchQueryPlugin
|           wraps the bool query in a script_score (its boosts)
|   ← (4) SymSearch HybridSearchQueryPlugin [after, sortOrder 500]
|
▼
Magento\OpenSearch\Model\OpenSearch::query()
|   ← (4b) SymSearch AddSearchPipelineParamPlugin [before]
|
▼
OpenSearch — hybrid query + jalabs_symsearch_hybrid search pipeline

- index side -
Magento\Elasticsearch\Model\Adapter\FieldMapper\Product\CompositeFieldProvider::getFields()
|   ← (1) SymSearch AddKnnFieldToMappingPlugin [after] → knn_vector mapping
Magento\Elasticsearch\Model\Adapter\Index\Builder::build()
|   ← (2) SymSearch EnableKnnIndexSettingPlugin [after] → index.knn + concurrent-search off
additionalFieldsProviderForElasticsearch (virtualType)
|   ← (3) SymSearch Embedding fields provider → injects vectors into docs

```

Because **(4)** is an *after* plugin on the **outer** `OpenSearch\...\Mapper`, it runs **after** Mirasvit's plugin (which acts on the *inner* `ElasticAdapter` `Mapper`). So when SymSearch captures the query body, Mirasvit's `script_score` boosting is already in it — and SymSearch puts that whole boosted query in as the keyword sub-query of the hybrid, preserving Mirasvit's ranking verbatim.

3. Component map

All under `app/code/JALabs/SymSearch/`.

Configuration & primitives

File	Responsibility
<code>Model/Config.php</code>	Typed reader for all <code>symsearch/*</code> config. <code>const FIELD_NAME = 'embedding_vector'</code> . <code>getModelVersion() = provider:model:dimensions</code> .
<code>Model/Config/Source/Provider.php</code>	Admin dropdown: openai / gemini / voyage.
<code>Model/VectorCodec.php</code>	<code>float[] ⇌ binary</code> . Packs little-endian float32 (<code>pack('g*')</code>) for compact <code>MEDIUMBLOB</code> storage.
<code>Exception/ProviderException.php</code>	Base for all embedding failures — the only thing callers must catch.
<code>Exception/RateLimitException.php</code>	Subclass carrying <code>getRetryAfterSeconds()</code> ; thrown on HTTP 429.

Embedding providers

File	Responsibility
Api/EmbeddingProviderInterface.php	<code>embed(string[] \$texts, int \$timeoutMs, string \$inputType): float[][]</code> . <code>TYPE_DOCUMENT</code> / <code>TYPE_QUERY</code> .
Model/Provider/OpenAiProvider.php	POST <code>api.openai.com/v1/embeddings</code> , Bearer auth, <code>dimensions</code> param, results sorted by <code>index</code> .
Model/Provider/VoyageProvider.php	POST <code>api.voyageai.com/v1/embeddings</code> , Bearer, <code>output_dimension</code> + <code>input_type</code> .
Model/Provider/GeminiProvider.php	POST <code>...:batchEmbedContents</code> , <code>x-goog-api-key</code> , <code>taskType</code> RETRIEVAL_DOCUMENT/QUERY, <code>outputDimensionality</code> . L2-normalizes each vector (Gemini truncated-dim vectors aren't unit length; the index uses inner-product space).
Model/Provider/ProviderResolver.php	Returns the configured provider from a DI-injected map.

All three providers wrap transport errors and JSON-parse errors in `ProviderException`, throw `RateLimitException` (with parsed retry hint) on 429, and validate response row shape — so `embed()`'s only failure mode is a `ProviderException` (or its subclass). That contract is what makes the keyword-fallback safe.

Storage & text

File	Responsibility
Model/Indexer/TextBuilder.php	Builds the embed text for a product from the configured attributes: source/select attributes by label, HTML stripped, whitespace collapsed, 4000-char cap.
Model/EmbeddingStorage.php	All DB access for both tables. Atomic batch claim (<code>SELECT ... FOR UPDATE</code> → mark <code>queued</code>), status transitions, hash-based vector dedup lookup/save, vector read-back for indexing, status counts, sweep helpers.

Async generation

File	Responsibility
<code>etc/communication.xml</code> , <code>queue_consumer.xml</code> , <code>queue_topology.xml</code> , <code>queue_publisher.xml</code>	RabbitMQ wiring for topic <code>jalabs.symsearch.embed</code> , consumer <code>jalabsSymsearchEmbed</code> .
Model/Queue/Dispatcher.php	Atomically claims <code>pending</code> items in 500-id messages and publishes them.
Model/Queue/Consumer.php	Per message: load products, build+hash text, dedup vs stored vectors, call provider in throttled+retried chunks, save vectors, mark <code>ready</code> . Catches any <code>\Throwable</code> → <code>markFailed</code> so a poison message can't loop forever.
Observer/MarkProductStaleObserver.php	<code>catalog_product_save_after</code> → mark the product <code>pending</code> .
Cron/Dispatch.php	every 5 min → dispatch pending per store.
Cron/Sweep.php	nightly → reset stale- <code>queued</code> , retry <code>failed</code> (≤ attempts cap), re-flag products whose <code>updated_at</code> changed (catches imports), seed missing rows. Each step isolated so one DB error doesn't abort the rest.

Index integration

File	Responsibility
Plugin/AddKnnFieldToMappingPlugin.php	Adds the <code>embedding_vector</code> <code>knn_vector</code> field to the index mapping (dimension from config; HNSW / Faiss / inner-product / fp16 scalar-quantization encoder).
Plugin/EnableKnnIndexSettingPlugin.php	Sets <code>index.knn=true</code> , <code>ef_search</code> , and <code>index.search.concurrent_segment_search.mode=none</code> (see §9).
Model/AdditionalFieldsProvider/Embedding.php	At reindex, reads <code>ready</code> vectors for the batch and injects <code>{embedding_vector: [...]}</code> into each product document. Reads DB only — never calls the API.

Query integration

File	Responsibility
<code>Service/QueryEmbedding.php</code>	Embeds the live query with Redis caching (14-day TTL, key = <code>sha1(modelVersion normalized)</code>), an 800ms budget, and <code>null</code> on any failure → caller falls back to keyword.
<code>Plugin/HybridSearchQueryPlugin.php</code>	The core rewrite (see §5).
<code>Plugin/AddSearchPipelineParamPlugin.php</code>	Adds <code>search_pipeline=jalabs_symsearch_hybrid</code> to the request when the body contains a <code>hybrid</code> query.
<code>Service/PipelineManager.php</code>	CRUD for the OpenSearch normalization search pipeline; verifies engine plugins.

CLI

`Console/Command/{GenerateCommand,StatusCommand,PipelineSetupCommand,RelevanceRunCommand}.php` — see USER-GUIDE §4.

4. Data model

jalabs_symsearch_item	jalabs_symsearch_vector
PK (entity_id, store_id)	PK (content_hash, model_version)
content_hash	content_hash
status pending queued ready failed	model_version provider:model:dims
attempts	vector MEDIUMBLOB (float32 LE)
updated_at (ON UPDATE)	created_at

- `item` tracks *what state each product/store-view is in*. One row per product per active store.
- `vector` stores the *actual embedding, once per unique text*. The join key is `content_hash` — so two products (or the same product in three store views) with identical embed-text share one vector row. This is the dedup that makes the backfill cheap.
- A model or dimension change bumps `model_version`, which makes every existing `vector` row a non-match → everything re-embeds on the next `--force` pass.

5. Query-time rewrite (HybridSearchQueryPlugin)

`afterBuildQuery(Mapper, array $searchQuery, RequestInterface $request):`

- Guards** (any fail → return the query untouched): - request name is `quick_search_container` (storefront quick search only), - sort is by relevance / `_score`, - `storefront_enabled` for the current store view, - query phrase ≥ 3 chars.
- Get the query text from the request tree**. It walks the `BoolExpression` (`should` / `must` / `mustNot`, depth-limited) for the first `MatchQuery` and reads `getValue()`. (Reading the HTTP `q` param via `QueryFactory` is the fallback — that param is empty in CLI/headless `SearchInterface` calls, which is why the request tree is the primary source.)
- Embed the query** via `QueryEmbedding` (cached). `null` → return untouched (keyword fallback).
- Build the hybrid**. The original (Mirasvit-boosted) query becomes sub-query 1; a `knn` over `embedding_vector` with `k` (or `min_score`) becomes sub-query 2: `json { "hybrid": { "queries": [ <original keyword query, untouched>, { "knn": { "embedding_vector": { "vector": [ ... ], "k": 100, "filter": { ... } } } ] } }`
- Copy structural filters into the knn branch**. Visibility / stock / website filters live in the bool query's `must` clause in this stack (Magento core and Mirasvit both emit there; `bool.filter` is empty). The plugin extracts the structural clauses (`term` / `terms` / `range` / `exists` / `ids`, recursively) from `must` and sets them as the knn `filter`. **Without this the vector branch would retrieve unfiltered and leak out-of-stock / catalog-only / out-of-website products into results.**
- Reduce the sort to `_score` only**. OpenSearch rejects a `hybrid` query if `_score` is combined with any other sort key, and Magento adds an `entity_id` tiebreaker. The plugin strips the sort to `_score` alone (relevance order is what hybrid produces anyway).
- The whole method is wrapped in a `\Throwable` catch → on any unexpected error it returns the original query (fallback), never an exception.

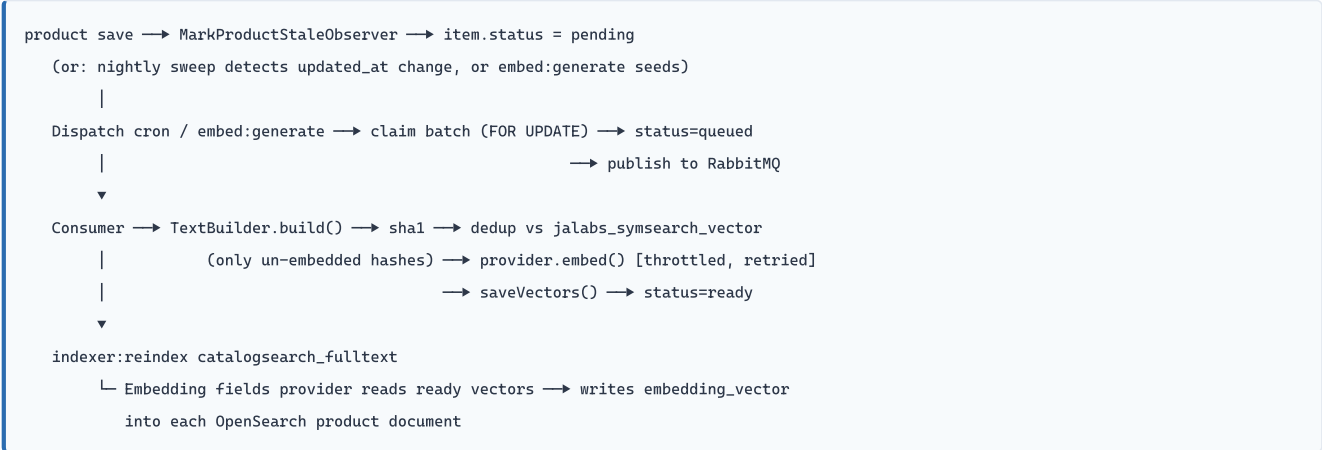
`AddSearchPipelineParamPlugin` then tags the request with `search_pipeline=jalabs_symsearch_hybrid` so OpenSearch applies the normalization processor.

The normalization pipeline

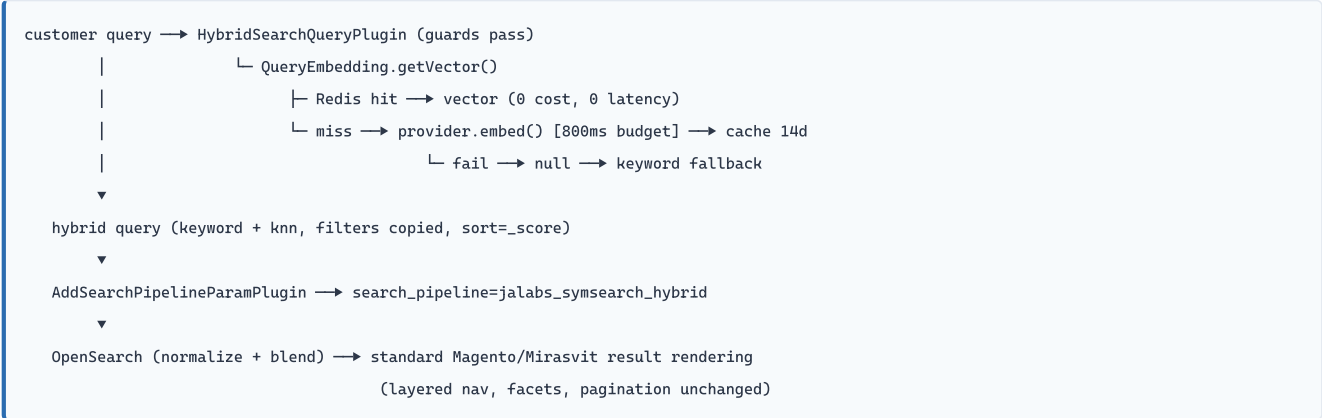
`PipelineManager` PUTs a search pipeline with a `normalization-processor`: `min_max` score normalization + `arithmetic_mean` combination, with weights derived from config and normalized to sum 1 (default `[0.7, 0.3]`). This is what makes the two sub-query scores — BM25 (unbounded) and cosine/IP (0–1) — comparable before they're blended. It is **cluster-global** (one pipeline), which is why the weight config is default-scope and requires `pipeline:setup` after a change.

6. Data flows end to end

Index time



Query time



7. Coexistence with Mirasvit Search Ultimate

Concern	How it's handled
Mirasvit's score boosts / rules	Preserved verbatim — the plugin runs <i>after</i> Mirasvit and embeds Mirasvit's whole <code>script_score</code> query as the hybrid's keyword sub-query.
Mirasvit autocomplete, misspell, landing pages	Untouched — SymSearch only rewrites the <code>quick_search_container</code> fulltext query on the results page.
Mirasvit's data-mapper / analyzer plugins	Independent extension points; SymSearch adds its own <code>knn_vector</code> field alongside them via the same <code>additionalFieldsProviderForElasticsearch</code> virtualType Magento core uses.
Mirasvit upgrades	No Mirasvit files touched. The one coupling is structural (it expects Mirasvit's <code>script_score</code> wrapper shape) — <code>HybridSearchQueryPlugin::extractFilter</code> handles both the plain <code>bool</code> and the <code>script_score.query.bool</code> shapes, and integration is plugin-only.

The division of labour: **Mirasvit owns keyword retrieval, ranking rules, synonyms, misspelling, indexing of the textual fields.** **SymSearch owns the vector field, the query embedding, and the hybrid blend.** They meet only at the OpenSearch query body.

8. Failure & degradation model

Semantic is always *additive*; every failure path degrades to keyword search.

Failure	Behaviour
Query embedding fails or exceeds 800ms	<code>QueryEmbedding</code> returns <code>null</code> → plugin returns the untouched keyword query. Customer sees normal results.
Product not yet embedded	No <code>embedding_vector</code> in its doc → the knn branch simply skips it; it can still match via BM25. Coverage grows asynchronously.
Provider 429 (rate limit)	<code>RateLimitException</code> with parsed retry hint → consumer waits and retries ($\leq 8\times$). Items stay <code>queued</code> , not failed.
Transient network blip	<code>ProviderException</code> (transport) → consumer retries ($\leq 3\times$, short backoff).
Repeated hard failure	Row marked <code>failed</code> after attempts; surfaced in <code>embed:status</code> ; nightly sweep retries up to the cap.
Malformed queue message	Logged and skipped (never crashes/poisons the consumer).
OpenSearch knn/pipeline/plugins missing	<code>pipeline:setup</code> reports it; <code>HybridSearchQueryPlugin</code> wraps everything in <code>\Throwable</code> → keyword fallback.
Module disabled	All plugins early-return on <code>isEnabled()</code> / <code>isStorefrontEnabled()</code> → stock Mirasvit behaviour.

9. Environment-specific issues (OpenSearch 3.3 / Magento internals)

Hard-won during build — relevant to anyone operating or extending this.

- OpenSearch adapter swallows query errors.** `Magento\OpenSearch\SearchAdapter\Adapter::query()` catches every exception, logs `report.CRITICAL`, and returns an `empty` result set. So a malformed query surfaces as "0 results", never an error. When debugging empty results, read `var/log/exception.log` for the real OpenSearch 4xx and `docker logs <opensearch>` for the Java stack.
- OpenSearch 3.3.2 + Lucene 10.3 concurrent-segment-search crash.** Freshly written segments crash on Mirasvit's case-insensitive wildcard clauses under concurrent segment search (`NFARunAutomaton` race → `array_index_out_of_bounds` / `Cannot read field "point" because "this.points[i]" is null`). This breaks **all** storefront search after any full reindex, independent of this module. Mitigation: `index.search.concurrent_segment_search.mode = none` on the product indices — set dynamically once, and **baked into** `EnableKnnIndexSettingPlugin` so every rebuild keeps it.
- Hybrid query ≠ multi-sort.** A `hybrid` query 400s if `_score` is combined with any other sort criterion; Magento adds an `entity_id` tiebreaker. Handled in §5 step 6.
- knn `k` vs `min_score` are mutually exclusive** — sending both is a 400. `HybridSearchQueryPlugin` sends one or the other.
- CLI/headless query text.** The HTTP `q` param doesn't exist in `SearchInterface` calls; the search phrase must be read from the request query tree (§5 step 2). A CLI `SearchInterface` search also needs an explicit relevance `SortOrder`, or OpenSearch 400s on a malformed `_script` sort.
- Gemini rate limits are per-text** (each text in a batch = one request) and **Gemini truncated-dim vectors aren't unit-normalized** — handled by the throttle/retry logic and the provider's L2-normalization respectively.

10. Testing

- Unit** (40 tests, mocked HTTP/DB): provider adapters incl. 429 / transport / malformed-row paths, vector codec round-trip, text builder, query-embedding cache + fallback, and the hybrid rewrite (filter extraction from `must`, `script_score` wrapper, sort reduction, `k` / `min_score` exclusivity, skip guards).
- Relevance suite** (`dev/symsearch/relevance-suite.csv`, `symsearch:relevance:run`): a curated set of natural-language EN, cross-language FR/AR, zero-result, and exact-keyword queries with expected SKUs. The regression gate for any ranking change.
- Live verification** done at rollout: vector coverage in the index, knn graph memory, storefront rendering with layered nav + pagination across EN/FR/AR, and the keyword-fallback path.

11. Extension points reference

If Mirasvit or Magento change, these are the seams to recheck:

SymSearch class	Hooks	Target
AddKnnFieldToMappingPlugin	after getFields	Magento\Elasticsearch\Model\Adapter\FieldMapper\Product\CompositeFieldProvider
EnableKnnIndexSettingPlugin	after build	Magento\Elasticsearch\Model\Adapter\Index\Builder
Embedding (fields provider)	getFields member	virtualType additionalFieldsProviderForElasticsearch
HybridSearchQueryPlugin	after buildQuery (sortOrder 500)	Magento\OpenSearch\SearchAdapter\Mapper
AddSearchPipelineParamPlugin	before query	Magento\OpenSearch\Model\OpenSearch
MarkProductStaleObserver	event	catalog_product_save_after

All registered in `etc/di.xml` / `etc/events.xml` / `etc/crontab.xml` .